

REPLIT PROMPT — YachtWorth: Yacht Photo Upload with Auto-Compression

CONTEXT

Update the photo section in AddYachtScreen.js (Section 6 — Photo). Replace the existing placeholder with a full photo upload implementation. Photos stored in Supabase Storage. Auto-compression applied before upload.

PACKAGES NEEDED

All already available in Expo — no new installations required:

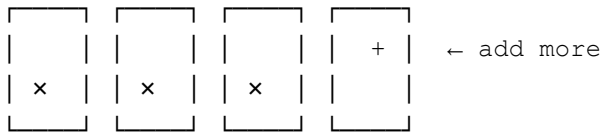
```
# Already in Expo SDK:
expo-image-picker      # camera + photo library
expo-image-manipulator # resize + compress
expo-file-system       # file handling
```

UI — PHOTO SECTION

Empty state (no photos yet):

```
[
  [
    [
      [📷]
    ],
    [large placeholder]
  ],
  [
    [No photos added yet]
  ],
  [
    [
      [📷] Take photo
    ],
    [
      [📖] Choose from library
    ]
  ]
]
```

With photos (up to 10):



Horizontal scroll row of thumbnails 80×80px

First photo = main/cover photo (star icon ★)

Tap thumbnail → full screen preview

Tap × → remove photo

Tap + → open source picker

Source picker (bottom sheet):

Add photo



Take a photo



Choose from library



Browse files

Cancel

PHOTO LIMITS

- Maximum photos per yacht: **10**
- Maximum file size after compression: **800 KB per photo**
- Accepted formats: JPG, PNG, HEIC (HEIC auto-converted to JPG)
- Minimum resolution: 800×600px (warn if smaller, still allow)
- First photo uploaded = cover photo (shown in fleet list card)

AUTO-COMPRESSION LOGIC

Apply automatically before every upload. User never sees this — it happens silently.

```
import * as ImageManipulator from 'expo-image-manipulator';

export async function compressPhoto(uri) {
  // Step 1: Get original dimensions
```

```

const original = await ImageManipulator.manipulateAsync(uri, [], { base64: false

// Step 2: Calculate target dimensions
// Max dimension: 1920px on longest side
// Maintain aspect ratio
const MAX_DIMENSION = 1920;
let { width, height } = original;

if (width > MAX_DIMENSION || height > MAX_DIMENSION) {
  if (width > height) {
    height = Math.round((height / width) * MAX_DIMENSION);
    width = MAX_DIMENSION;
  } else {
    width = Math.round((width / height) * MAX_DIMENSION);
    height = MAX_DIMENSION;
  }
}

// Step 3: Compress
const compressed = await ImageManipulator.manipulateAsync(
  uri,
  [{ resize: { width, height } }],
  {
    compress: 0.75,          // 75% quality - good balance size/quality
    format: ImageManipulator.SaveFormat.JPEG,
    base64: false,
  }
);

// Step 4: Check if still too large, compress more aggressively
const fileInfo = await FileSystem.getInfoAsync(compressed.uri, { size: true });
if (fileInfo.size > 800 * 1024) { // > 800KB
  const recompressed = await ImageManipulator.manipulateAsync(
    compressed.uri,
    [],
    {
      compress: 0.55,        // 55% quality if still too large
      format: ImageManipulator.SaveFormat.JPEG,
      base64: false,
    }
  );
  return recompressed.uri;
}

return compressed.uri;
}

```

Progress indicator while compressing:

Optimising photo...  80%

Small text below the photo thumbnail while uploading.

UPLOAD TO SUPABASE STORAGE

```
import * as FileSystem from 'expo-file-system';
import { supabase } from '../utils/supabase';

export async function uploadYachtPhoto(localUri, yachtId, index) {
  // 1. Compress first
  const compressedUri = await compressPhoto(localUri);

  // 2. Read file
  const fileExt = 'jpg';
  const fileName = `${yachtId}/photo_${index}_${Date.now()}.${fileExt}`;

  // 3. Upload to Supabase Storage bucket 'yacht-photos'
  const formData = new FormData();
  formData.append('file', {
    uri: compressedUri,
    name: fileName,
    type: 'image/jpeg',
  });

  const { data, error } = await supabase.storage
    .from('yacht-photos')
    .upload(fileName, formData, {
      contentType: 'image/jpeg',
      upsert: true,
    });

  if (error) throw error;

  // 4. Get public URL
  const { data: urlData } = supabase.storage
    .from('yacht-photos')
    .getPublicUrl(fileName);

  return urlData.publicUrl;
}
```

SUPABASE STORAGE SETUP

Create bucket if not exists:

```
-- In Supabase dashboard or via API:
-- Bucket name: 'yacht-photos'
-- Public: true (photos are viewable without auth)
-- File size limit: 5MB (raw, before compression)
-- Allowed MIME types: image/jpeg, image/png, image/heic
```

Update yachts table:

```
-- Already has photo_url (single) - extend to support multiple
alter table yachts add column if not exists photo_urls jsonb default '[]'::jsonb;
alter table yachts add column if not exists cover_photo_url text;

-- cover_photo_url = first photo, used in fleet cards
-- photo_urls = array of all photo URLs ["url1", "url2", ...]
```

PERMISSION HANDLING

Request permissions gracefully before opening camera or library:

```
// Camera permission
const { status: cameraStatus } = await ImagePicker.requestCameraPermissionsAsync()
if (cameraStatus !== 'granted') {
  Alert.alert(
    'Camera access needed',
    'Please allow camera access in Settings to take photos.',
    [
      { text: 'Cancel', style: 'cancel' },
      { text: 'Open Settings', onPress: () => Linking.openSettings() }
    ]
  );
  return;
}

// Photo library permission
const { status: libraryStatus } = await ImagePicker.requestMediaLibraryPermission
if (libraryStatus !== 'granted') {
  Alert.alert(
```

```
    'Photo library access needed',
    'Please allow photo library access in Settings.',
    [
      { text: 'Cancel', style: 'cancel' },
      { text: 'Open Settings', onPress: () => Linking.openSettings() }
    ]
  );
  return;
}
```

TAKE PHOTO (camera):

```
const result = await ImagePicker.launchCameraAsync({
  mediaTypes: ImagePicker.MediaTypeOptions.Images,
  allowsEditing: true,          // crop before adding
  aspect: [16, 9],             // 16:9 crop for yacht photos
  quality: 1,                  // full quality – we compress ourselves
  exif: false,                  // strip EXIF data (privacy)
});

if (!result.canceled) {
  await handlePhotoAdded(result.assets[0].uri);
}
```

CHOOSE FROM LIBRARY:

```
const result = await ImagePicker.launchImageLibraryAsync({
  mediaTypes: ImagePicker.MediaTypeOptions.Images,
  allowsMultipleSelection: true, // pick up to 5 at once
  selectionLimit: 5,
  quality: 1,
  exif: false,
});

if (!result.canceled) {
  for (const asset of result.assets) {
    await handlePhotoAdded(asset.uri);
  }
}
```

HANDLE PHOTO ADDED (full flow):

```
async function handlePhotoAdded(uri) {
  if (photos.length >= 10) {
    Alert.alert('Maximum reached', 'You can add up to 10 photos per yacht.');
```

return;

```
  }

  // Show loading state for this slot
  setLoadingPhoto(true);

  try {
    // 1. Compress
    const compressedUri = await compressPhoto(uri);

    // 2. Add to local state immediately (show thumbnail)
    const tempId = Date.now().toString();
    setPhotos(prev => [...prev, { id: tempId, uri: compressedUri, uploading: true

    // 3. Upload to Supabase (if yacht already saved)
    if (yachtId) {
      const url = await uploadYachtPhoto(compressedUri, yachtId, photos.length);
      setPhotos(prev => prev.map(p =>
        p.id === tempId ? { ...p, uploading: false, url } : p
      ));
    }

    // If yacht not saved yet, URLs will be uploaded when form is saved

  } catch (error) {
    Alert.alert('Upload failed', 'Could not add photo. Please try again.');
```

setPhotos(prev => prev.filter(p => p.id !== tempId));

```
  } finally {
    setLoadingPhoto(false);
  }
}
```

PHOTO THUMBNAIL COMPONENT

```
// PhotoThumbnail.jsx
// Props: uri, url, uploading, isCover, onRemove, onSetCover, onPreview

// States:
// uploading = true → show spinner overlay + "Optimising..."
```

```
// isCover = true → show gold ★ badge top-left
// Normal → show ✕ button top-right

// Long press → context menu: [Set as cover] [Remove]
// Tap → full screen preview
```

FULL SCREEN PREVIEW

```
// PhotoPreview modal
// Swipe between photos
// Double tap to zoom
// Share button (top right)
// Delete button (bottom)
// "Set as cover" button (bottom)
```

FILE STRUCTURE

```
/components/MyYacht/
  PhotoSection.js      ← main photo section component
  PhotoThumbnail.js    ← single photo thumbnail
  PhotoPreviewModal.js ← full screen preview
  PhotoSourceSheet.js  ← bottom sheet: camera/library/files

/utils/
  photoCompression.js  ← compressPhoto() function
  photoUpload.js       ← uploadYachtPhoto() function
```

ERROR STATES

Situation	Message shown
No camera permission	"Allow camera access in Settings" + button
No library permission	"Allow photo library access in Settings" + button
Upload failed	"Upload failed. Tap to retry." on thumbnail
File too large (>10MB raw)	"File too large. Please choose a smaller image."

Wrong format	"Only JPG and PNG photos are supported."
Max 10 reached	"Maximum 10 photos per yacht."
No internet during upload	Photos saved locally, sync when online

OFFLINE HANDLING

If no internet connection during upload:

- Save compressed photo to local AsyncStorage temporarily
- Show "Saved locally — will sync when online" indicator
- When connection restored → auto-upload pending photos
- Show sync indicator in fleet card until all photos uploaded

PRIORITY ORDER

1. `photoCompression.js` — `compressPhoto()` function
2. `PhotoSourceSheet.js` — camera / library / files picker
3. Camera + library permission requests
4. `PhotoThumbnail.js` component
5. `PhotoSection.js` — grid with add button
6. Supabase Storage upload
7. `PhotoPreviewModal.js` — full screen view
8. Cover photo logic
9. Offline handling

IMPORTANT NOTES

- Compression is SILENT — no progress bar for user, just a brief spinner on the thumbnail

- EXIF data stripped for privacy (location data removed automatically)
 - 16:9 crop enforced for camera — yacht photos look better in landscape
 - Library picker: allow multiple selection up to 5 at once
 - Cover photo = first photo in array by default, user can change it
 - When yacht deleted → delete all photos from Supabase Storage bucket too
 - Photo URLs saved in `yachts.photo_urls` (array) and `yachts.cover_photo_url` (first photo)
-

YachtWorth — My Yacht: Photo Upload Add to AddYachtScreen.js Section 6 Compression target: max 800KB per photo, max 1920px longest side, 75% JPEG quality